

Projet Site Web R3st0.fr

Itération n°2

Ticket n°3 : Identification de l'équipe de développement en pied de page

1) Diagnostic

Quand j'ai affiché la page, le texte « Mise à jour effectuée par équipe n°7 » apparaissait collé à gauche.

J'ai d'abord pensé que c'était un problème d'HTML, mais en regardant le code j'ai vu que le fichier CSS était inclus avec `require_once`. Ou, `require_once` sert pour du PHP, pas pour charger une feuille de style. Donc le navigateur ne recevait jamais les règles CSS → d'où le texte n'était pas centré

2) Localisation

Le souci se trouve dans la partie du code qui affiche la liste des restaurants (fichier `vue/listeRestos.php`).

La ligne fautive est celle qui tente d'inclure "`$racine/css/ref.css`" avec `require_once`.

3) Modifications envisagées

Pour corriger le problème, il faudra :

- remplacer le `require_once` par une balise `<link>` qui permet réellement de charger du CSS dans le navigateur ;
- créer un fichier `ref.css` dans le dossier `css/` et y mettre la règle pour centrer le paragraphe (via `text-align: center`).

Ce choix est justifié car c'est la méthode standard reconnue par tous les navigateurs, contrairement à l'inclusion PHP qui n'est pas adaptée.

4) Tests prévus

Une fois les changements faits, je testerai :

- que la page affiche bien le texte centré ;
- que l'URL du fichier `css/ref.css` est accessible directement dans le navigateur ;
- que les autres pages ne sont pas impactées par ce changement.

Afin d'identifier notre équipe en pied de page je j'ai ajouté un head pour importer le CSS.

```
<head>
  <meta charset="utf-8">
  <title><?= $titre ?></title>
  <link rel="stylesheet" type="text/css" href="./css/ref.css">
</head>
```

Puis j'ai créé une balise `<p>` pour ajouter du texte.

```
<div>
  <p id="ref"> Mise a jour effectuée par équipe n°7</p>
</div>
```

Et enfin j'ai fait un code CSS pour positionner et mettre en forme le texte

```
#ref {
  text-align: right; /* alligne le texte a droite */
  margin-top: 15px; /* Un petit espace au-dessus */
  margin-right: 20px; /* Un petit espace a droite*/
  font-size: 14px; /* Taille du texte adaptée */
  color: #333; /* Couleur du texte (optionnel) */
}
```

Voici le rendu :

Accueil
Recherche
★★★★★
r3st0.fr
CGU
Rapport

Decouvrez les meilleurs restaurants avec resto.fr

Top 4 des meilleurs restaurants



[L'entrepote](#)
2 rue Maurice Ravel
33000 Bordeaux



[Cidrerie du fronton](#)
Place du Fronton
64210 Arbonne



[le bar du charcutier](#)
30 rue Parlement Sainte-Catherine
33000 Bordeaux



[la petite auberge](#)
15 rue des cordeliers
64100 Bayonne

Classement basé sur les critiques de nos utilisateurs.

Mise a jour effectuée par équipe n°7

Ticket n°4 :

. La fonction `ControleurPrincipal($action)` renvoie la page par défaut quand l'action est inconnue, mais sans appeler la fonction `AjouterMessage()`. L'utilisateur n'est donc pas informé

. Le fichier à modifier `includes/controleurPrincipal.inc.php`, et la méthode à utiliser est la méthode `controleurPrincipal($action)`

. Pour ajouter le message d'erreur, il faut simplement ajouter cette ligne dans le code, qui appelle la méthode pour ajouter un message d'erreur (`ajouterMessage()`), par rapport à une action qui est fausse (le tout dans le fichier `controleurPrincipal.inc.php`) :

```
else{
    ajouterMessage("La page $action n'existe pas.");
    return $lesActions["defaut"];
}
```

. Et voici le message d'erreur sur la page (avec action=noahguibert) :



Ticket n°5 : amélioration du chiffrement des mots de passes

Dans un premier temps on doit gérer l'enregistrement des mots de passe pour ça on doit modifier la fonction `updateMdp()` dans le fichier "UtilisateurDAO.class.php".

On remplace la méthode `crypt()` par la méthode `password_hash()`.

```

136 public static function updateMdp(int $idU, string $mdpClair): bool {
137     $ok = false;
138     try {
139         $requete = "UPDATE utilisateur SET mdpU = :mdpU WHERE idU = :idU";
140         $stmt = Bdd::getConnexion()->prepare($requete);
141
142         // Hachage sécurisé avec password_hash
143         $mdpHash = password_hash($mdpClair, PASSWORD_DEFAULT);
144
145         $stmt->bindValue(':mdpU', $mdpHash, PDO::PARAM_STR);
146         $stmt->bindValue(':idU', $idU, PDO::PARAM_INT);
147
148         $ok = $stmt->execute();
149     } catch (PDOException $e) {
150         throw new Exception("Erreur dans la méthode " . get_called_class() . "::updateMdp : <br/>" . $e->getMessage());
151     }
152     return $ok;
153 }
154
155

```

Ensuite j'ai modifié la méthode `login()` dans le fichier "authentification.inc.php" pour que lorsqu'on se connecte on utilise la méthode "password_verify" cette méthode compare le mot de passe

entré par l'utilisateur et le mot de passe crypté dans base de données et renvoie un booléen.

```

17 function login(string $mailU, string $mdpU): void {
18     if (!isset($_SESSION)) {
19         session_start();
20     }
21
22     // Rechercher les données relatives à cet utilisateur
23     $util = UtilisateurDAO::getOneByMail($mailU);
24
25     // Si l'utilisateur est connu (mail trouvé dans la BDD)
26     if (!is_null($util)) {
27         $mdpBD = $util->getMdpU(); // hash stocké en BDD
28         $idU   = $util->getIdU();
29
30         // Si l'utilisateur est connu (mail trouvé dans la BDD)
31         if (password_verify($mdpU, $mdpBD) == true) {
32             $_SESSION["idU"] = $idU;
33             $_SESSION["mailU"] = $mailU;
34             $_SESSION["mdpU"] = $mdpU;
35         } elseif (trim($mdpBD) == trim(crypt($mdpU, $mdpBD))) {
36             // le mot de passe est celui de l'utilisateur dans la base de données
37             $_SESSION["idU"] = $idU;
38             $_SESSION["mailU"] = $mailU;
39             $_SESSION["mdpU"] = $mdpU;
40         }
41     }
42 }

```

Ticket n°6 :

. La requête SQL pour appeler le top 4 des meilleurs restaurant contient une erreur, ce qui n'affiche donc pas les meilleurs restaurant dans l'accueil, mais ceux avec le plus de note ($5/5 + 4/5 = 9$).

. Le fichier concerné est le fichier RestoDAO.class.php, ligne 83, et la méthode concernée est la méthode getTop4()

. Pour pouvoir afficher les meilleurs restos, on va modifier la requête SQL, plus exactement la partie SELECT COUNT(note), et on va remplacer le COUNT() par AVG(), une fonction permettant de calculer la moyenne des notes, et non de les

additionner :

```
$requete = "SELECT AVG(note) AS NotesCumulees, r.idR, nomR, numAdrR, voieAdrR, cpR, villeR, latitudeDegR, longitudeDegR, descR, horaireR
FROM resto r
INNER JOIN critiquer c ON r.idR = c.idR
GROUP BY r.idR, nomR, numAdrR, voieAdrR, cpR, villeR, latitudeDegR, longitudeDegR, descR, horaireR
ORDER BY NotesCumulees DESC
LIMIT 4;
```

Et voici la bonne liste du top 4 des restaurants :



Ticket n°7 :

. Ici, on doit simplement ajouter une ligne echo pour pouvoir afficher la note, grâce à la variable noteMoy qui va me permettre d'afficher sur la page la note moyenne

. Les fichiers que je vais utiliser seront le fichier vueDetailResto.php, ligne 39 (à ajouter), et le fichier detailResto.php, et je vais utiliser la variable noteMoy (qu'on modifiera juste après pour une question d'arrondissement de valeur), qui contient déjà la moyenne des notes, pour chaque page.

. Tout d'abord, on va créer une nouvelle variable noteMoyBrut dans le fichier detailResto.php, qui fonctionnera comme noteMoy, mais qui évitera d'arrondir la note (pour avoir les décimales), avec ce code ci-dessous :

```
$noteMoy = round(CritiqueDAO::getNoteMoyenneByIdR($idR), 0);
$noteMoyBrut = CritiqueDAO::getNoteMoyenneByIdR($idR);
```

Ensuite, dans le fichier `vueDetailResto.php`, on va ajouter ce code (ligne 39) pour afficher sur la page la moyenne à côté des têtes de bonhommes :

```

38 <span id="note">
39     <?php echo $noteMoyBrut ?>
40     <?php for ($i = 1; $i <= 5; $i++) { ?>

```

. Voici ce que ça affiche lorsqu'on regarde un des restaurants :

la petite auberge ★



4.5 🍷 🍷 🍷 🍷 🍷

description

Adresse

15 rue des cordeliers
64100 Bayonne

Photos



Critiques

@lex
4/5 Bon accueil.

pich

Nico40
5/5 Excellent.

Ticket n°8 :

Pour cette partie, j'ai ajouté une vérification sur le champ code postal lors de la recherche d'un restaurant par adresse.

Avant la modification, le formulaire n'effectuait aucun contrôle : l'utilisateur pouvait saisir du texte ou un code incomplet, ce qui pouvait provoquer une recherche incohérente ou vide.

J'ai donc ajouté une validation côté serveur dans le contrôleur rechercheResto.php. Après la récupération du champ \$_POST["cpR"], j'ai utilisé une expression régulière pour vérifier que la saisie correspond bien à exactement cinq chiffres.

Si ce n'est pas le cas, un message d'erreur est ajouté dans un tableau \$erreurs

```
if ($cpR !== "" && !preg_match('/^[0-9]{5}$/', $cpR)) {
    $erreurs[] = "le code postal saisi n'est pas valide";
}
```

Ensuite, j'ai modifié la fin du contrôleur afin que, si des erreurs sont détectées, la page de recherche (vueRechercheResto.php) soit réaffichée avec le message d'erreur, au lieu d'afficher la liste des restaurants.

Cela permet de bloquer la recherche tant que la saisie n'est pas correcte.

```
<?php if (!empty($erreurs)): ?>
    <div id="error">
        Liste des erreurs
    </div>
    <ul style="margin-top:0; padding-left:20px; color:#333;">
        <?php foreach ($erreurs as $msg): ?>
            <li><?= htmlspecialchars($msg, ENT_QUOTES, 'UTF-8') ?></li>
        <?php endforeach; ?>
    </ul>
<?php endif; ?>
```

Grâce à ces modifications, la page affiche désormais un **message d'erreur clair** en cas de code postal invalide, tout en conservant la valeur saisie.

Cette solution améliore la fiabilité du formulaire et l'expérience utilisateur, car elle évite les recherches inutiles et guide clairement l'utilisateur vers une saisie correcte.

Diagnostic

Lors de la recherche d'un restaurant par adresse, le champ **code postal** n'était soumis à aucun contrôle.

L'utilisateur pouvait donc saisir n'importe quelle valeur (texte, code incomplet, trop

long...), ce qui provoquait des résultats incohérents et un manque de fiabilité dans les recherches.

Le problème venait d'une **absence de validation côté serveur** dans le contrôleur `rechercheResto.php` et d'un **manque de retour visuel** dans la vue `vueRechercheResto.php`.

📍 Localisation

- **Fichier : `contrôleur/rechercheResto.php`**
 - Insertion du code de validation après la récupération du champ `$_POST["cpR"]` (lignes ~45).
 - Modification du bloc conditionnel d'affichage des vues en bas du contrôleur (lignes ~80–85).
- **Fichier : `vue/vueRechercheResto.php`**
 - Ajout d'un bloc d'affichage des erreurs avant le formulaire `<form>`.
 - Mise à jour de l'input `cpR` pour conserver la valeur saisie et ajouter des attributs de validation HTML.

Ticket n°9 :

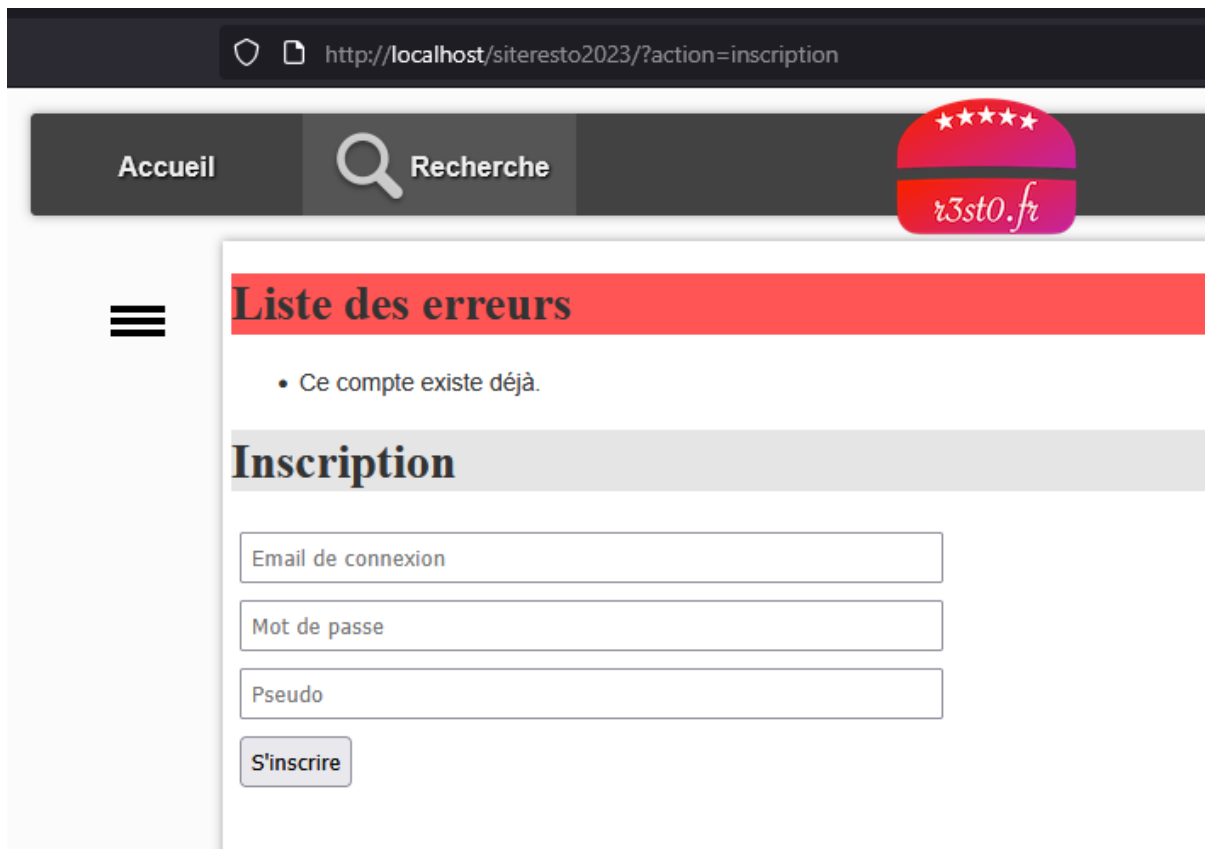
. Le code n'avait pas d'insertion de message d'erreur sur les emails déjà existante, donc il faut rajouter le code de ce dernier

. Le fichier modifier est le fichier `inscription.php`, au niveau de la ligne 22 (là où tous les messages d'erreurs sont répertoriés pour que le programme puisse marcher sur la page), où on va appeler la variable `$mailU`, et la méthode `ajouterMessage()`

. Pour cela, on va simplement ajouter la méthode ajouterMessage(), et appelé la variable pour justifier la cause du message d'erreur :

```
// vérification si l'email existe déjà ?
$utilisateurExistant = UtilisateurDAO::getOneByMail($mailU);
if ($utilisateurExistant !== null) {
    // message d'erreur si l'adresse est déjà utilisée
    ajouterMessage("Ce compte existe déjà.");
} else {
```

. Et voici le message d'erreur :



Ticket n°10 :

1) Diagnostic (ce qui posait problème)

Les champs de recherche (ex. nomR, voieAdrR, cpR, villeR) pouvaient être concaténés dans des requêtes SQL → risque d'injection (' OR 1=1 --, etc.).

Erreur vue côté DAO : « pending result sets... closeCursor() » → certains PDOStatement n'étaient pas libérés avant une autre requête (boucles qui appellent d'autres DAO, etc.).

2) Localisation

Méthodes de filtre dans modele/dao/RestoDAO.class.php :

- getAllByNomR(\$nomR)
- getAllByAdresse(\$voieAdrR, \$cpR, \$villeR)
- getAllMultiCriteres(...)

Appelé depuis controleur/rechercheResto.php.

Modifications apportées

1. Sécurisation des requêtes SQL

Toutes les requêtes ont été converties en requêtes préparées à l'aide de PDO.

Chaque paramètre utilisateur est désormais lié à une variable via bindValue() afin d'empêcher toute injection.

De plus, j'ai ajouté une méthode interne pour échapper les caractères spéciaux utilisés dans les clauses LIKE (% , _).

```
private static function escLike(string $s): string {
    // échappe % et _ pour LIKE
    return str_replace(['\\', '%', '_'], ['\\\\', '\\%', '\\_'], $s);
}
```

```
public static function getAllByNomR(?string $nomR): array {
    $pdo = Bdd::getPdo();
    $sql = "SELECT * FROM resto";
    $params = [];
    if ($nomR !== null && $nomR !== '') {
        $sql .= " WHERE nomR LIKE :nom ESCAPE '\\'";
        $params[':nom'] = self::escLike($nomR) . '%';
    }
    $stmt = $pdo->prepare($sql);
    foreach ($params as $k => $v) { $stmt->bindValue($k, $v, PDO::PARAM_STR); }
    $stmt->execute();
    $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
    $stmt->closeCursor();

    return self::rowsToRestos($rows); // adapte au mapping existant
}
```

```

public static function getAllByAdresse(?string $voieAdrR, ?string $cpR, ?string $villeR): array {
    $pdo = Bdd::getPdo();
    $where = [];
    $params = [];

    if ($voieAdrR !== null && $voieAdrR !== '') {
        $where[] = "voieAdrR LIKE :voie ESCAPE '\\\\'";
        $params[':voie'] = self::escLike($voieAdrR) . '%';
    }

    if ($cpR !== null && $cpR !== '') {
        $where[] = "cpR = :cp";
        $params[':cp'] = $cpR;
    }

    if ($villeR !== null && $villeR !== '') {
        $where[] = "villeR LIKE :ville ESCAPE '\\\\'";
        $params[':ville'] = self::escLike($villeR) . '%';
    }

    $sql = "SELECT * FROM resto";
    if (!empty($where)) {
        $sql .= " WHERE " . implode(" AND ", $where);
    }
    $stmt = $pdo->prepare($sql);
    foreach ($params as $k => $v) { $stmt->bindValue($k, $v, PDO::PARAM_STR); }
    $stmt->execute();
    $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
    $stmt->closeCursor();
    return self::rowsToRestos($rows);
}

```

Jeu d'essai :

Cas de test	Saisie	Résultat attendu	Résultat obtenu
Recherche normale	nomR = "Pizza"	Résultats correspondant au mot-clé	Conforme
Tentative d'injection SQL	nomR = ' OR 1=1	Aucun élargissement des résultats, requête filtrée	Conforme
Injection sur ville	villeR = Nantes' OR 'x'='x	Filtrée, aucun comportement anormal	Conforme
Caractères spéciaux % ou _	voieAdrR = %	Interprété comme texte, pas comme wildcard SQL	Conforme
Code postal invalide	cpR = 44A00	Message d'erreur "le code postal saisi n'est pas valide"	Conforme
Recherche combinée	Nom + Ville	Requête exécutée normalement, pas d'erreur	Conforme

Ticket n°11 :

. La recherche en mutli-critère est bien présente sur le site, mais ne fonctionne pas, dû à des oublies et des erreurs sur le code de la méthode getAllMultiCriteres.

. Le fichier à modifier est le fichier RestoDAO.class.php, au niveau de la ligne 182, la méthode getAllMultiCriteres aura donc des modifications à apporter.

. Tout d'abord, certaines lignes comme l'initialisation de la variable \$filtre ne marche pas, on va donc enlever cette partie (il faudra donc modifier la requête SQL puisqu'elle utilisait la variable \$filtre)

```
public static function getAllMultiCriteres(string $extraitNomR, string $voieAdrR, string $cpR, string $villeR, array
    $lesObjets = array();
    try {
        //ligne initialisation $filtre (de tri) est inutile, elle ne servira pas à grand chose ici
        $requete = "SELECT DISTINCT r.* "
            . " FROM resto r "
```

Ensuite, il y a un inner join inutile dans la requête, il faut donc l'enlever, et également modifier la requête pour qu'elle puisse fonctionner correctement :

```
try {
    //ligne initialisation $filtre (de tri) est inutile, elle ne servira pas à grand
    $requete = "SELECT DISTINCT r.* "
        . " FROM resto r "
        //inner join inutile (ligne enlevée)
        . " WHERE (:nomR IS NULL OR r.nomR LIKE :nomR) "
        . " AND (:voieAdrR IS NULL OR r.voyeAdrR LIKE :voieAdrR) "
        . " AND (:cpR IS NULL OR r.cpR LIKE :cpR) "
        . " AND (:villeR IS NULL OR r.villeR LIKE :villeR) "
        . " ORDER BY r.nomR";
```

. Si on recherche un restaurant avec comme rue “Place [...]”, voici le résultat :

Recherche d'un restaurant

Recherche multi-critères

nom du restaurant	Place
code postal	ville

http://localhost/siteresto2023/?action=recherche&critere=multi

Accueil Recherche 

Liste des restaurants



[Cidrerie du fronton](#)
Place du Fronton
64210 Arbonne

Si on recherche avec le nom “entrepote” :

The image shows two screenshots of a web application interface. The top screenshot displays the search form titled "Recherche d'un restaurant". The bottom screenshot displays the search results titled "Liste des restaurants".

Top Screenshot: Recherche d'un restaurant

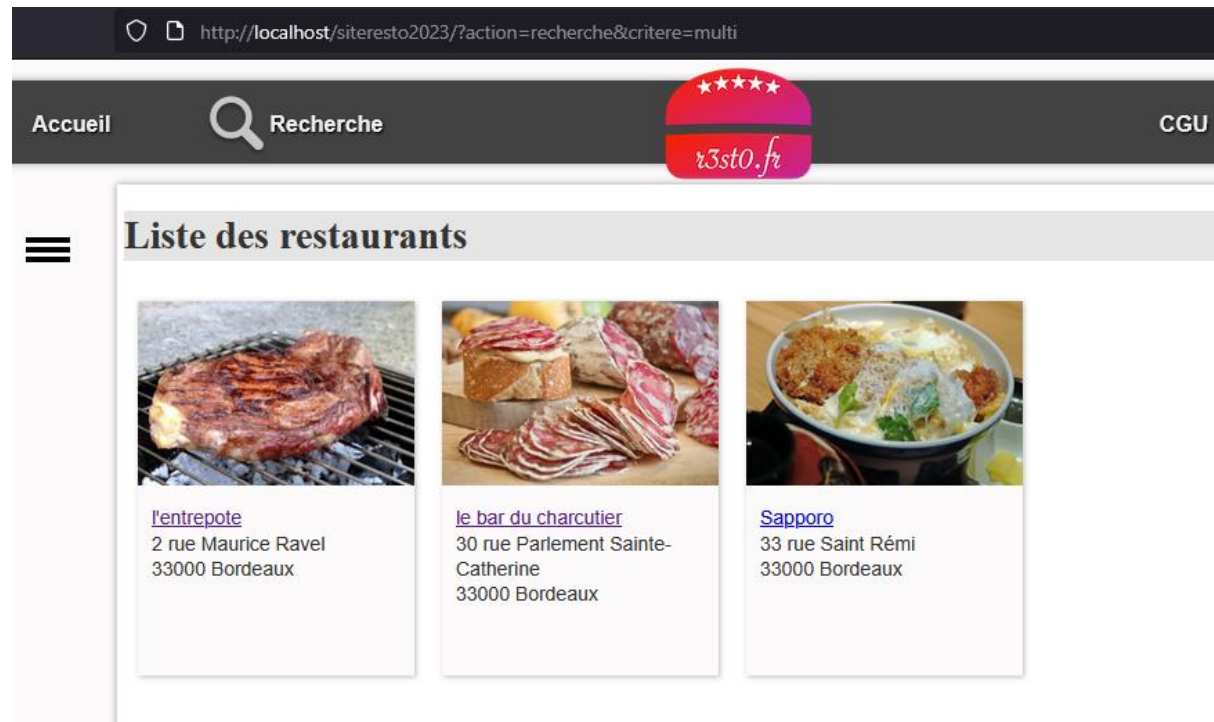
The interface includes a navigation bar with "Accueil", "Recherche", and "CGU" links, and a logo for "r3st0.fr" with five stars. A hamburger menu is visible on the left. The search form is titled "Recherche multi-critères" and contains four input fields:

- Search term:
- Street:
- Postal code:
- City:

Bottom Screenshot: Liste des restaurants

The search results are displayed under the heading "Liste des restaurants". A single result is shown with a photo of a roasted meat dish on a grill. Below the photo, the restaurant name "l'entrepote" is listed, followed by its address: "2 rue Maurice Ravel" and "33000 Bordeaux".

Et enfin, si on recherche avec la ville Bordeaux et le nom “rue [...]” dans la rue, on obtient ceci :



DAILY SCRUM

29/09/2025 :

La semaine dernière, Garence a fini le ticket numéro 1, Romain a fini les partie I, II, et IV du ticket numéro 2, et Noah a fini la partie III du ticket numéro 2.

Aujourd’hui, chacun fait 1 ticket par heure.